14 OpenClaw AI Agents Running 100% Locally

on a Single RTX 3090 GPU

52ms, Not 6 Minutes

Zero cloud APIs. Zero subscriptions. Full reasoning on one GPU from 2020.

6,667x faster with KV cache offloading

Andrew Darius
AIgenteur Academy
skool.com/aigenteur-academy-9764

A home computer from 2019 with an RTX 3090 GPU from 2020 - running **Qwen3.5** models released this year. No cloud API. No subscription. Everything runs on one machine.

	Without Optimization	With KV Cache + Agent Routing
200K context prompt	5.8 minutes (9B)	52 ms (9B)
200K context prompt	2.4 minutes (2B)	21 ms (2B)
Speedup		6,667× faster

> *Same GPU. Same model. Just smarter caching. The KV cache stores the system prompt so it's only processed once - every subsequent request skips straight to the new tokens.*

The Problem

We have **14 AI agents** with very different needs:

- Some need **deep reasoning** (strategy, compliance, code architecture)
- Some need **raw speed** (trend scouting, short copy, voice workflows)

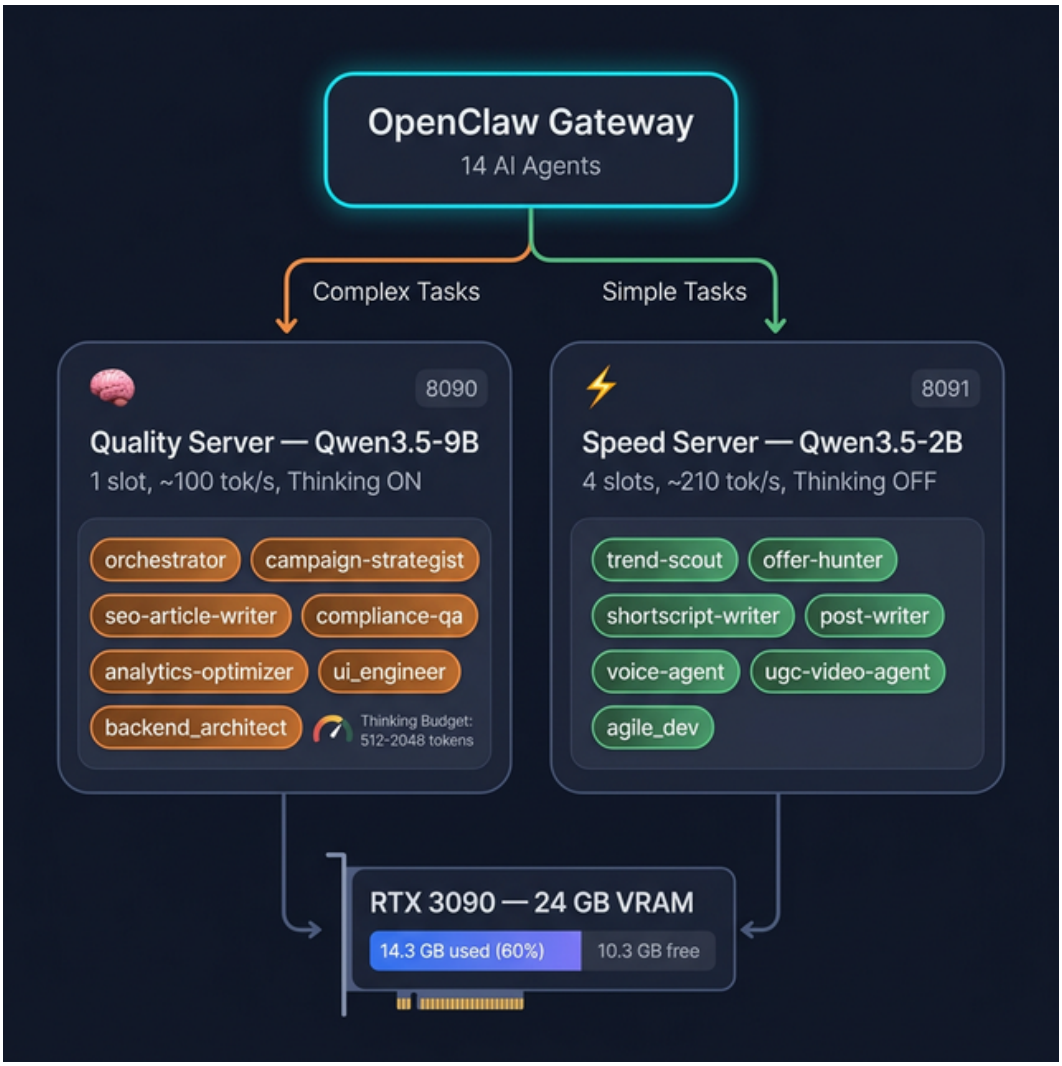
Running all 14 on a single large model wastes GPU on simple tasks. Running them all on a small model degrades quality on complex ones.

The Solution: Two Models, One GPU

We run **two LLM servers simultaneously** on a single RTX 3090 (24 GB), each optimized for its role:

	Quality Server	Speed Server
Model	Qwen3.5-9B	Qwen3.5-2B
Port	8090	8091
Thinking	ON (budgeted)	OFF
Slots	1 (serial)	4 (parallel)
Context	262K tokens	262K tokens
Speed	~100 tok/s	~210 tok/s
VRAM	~10.4 GB	~4.0 GB
Total	14.3 GB used	10.3 GB free

Architecture Overview



OpenClaw routes complex tasks to the 9B server and simple tasks to the 2B server

What is "Thinking" and Why Budget It?

Qwen3.5 models can **reason internally** before answering. The model generates hidden "thinking tokens" inside `<think>...</think>` tags, then produces the actual answer.

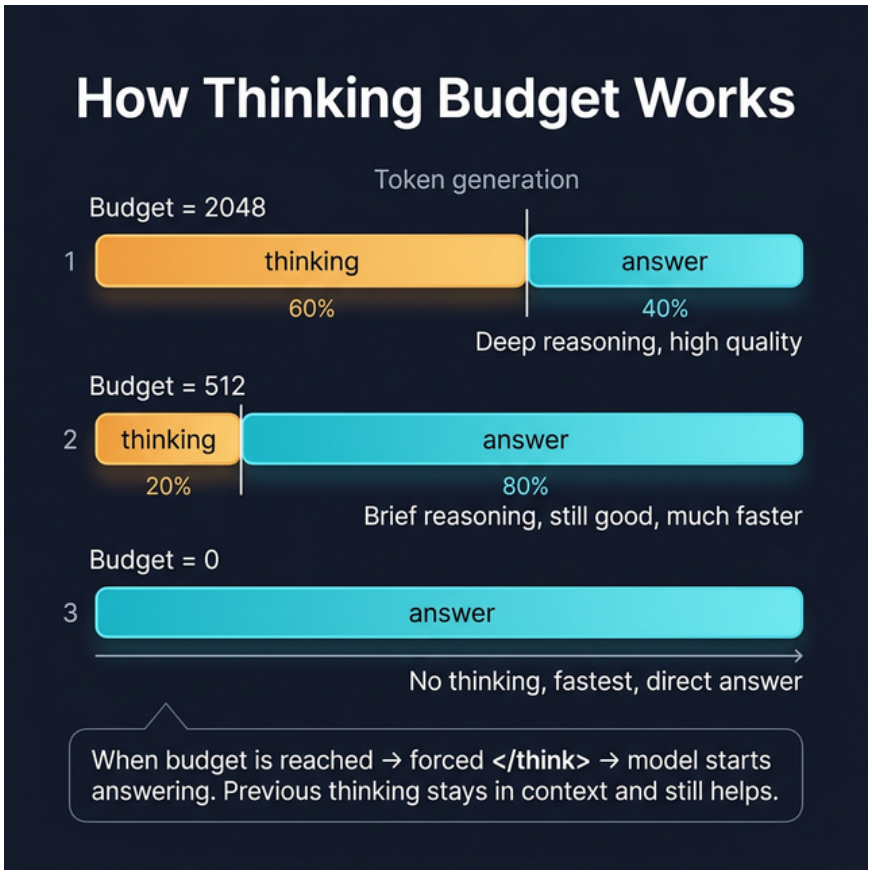
Example:

```
User: "Review this ad for compliance issues" Model thinking (hidden): 1. Check claims...
"50% off" needs substantiation 2. Check disclaimers... missing terms link Model answer
(visible): "Two issues found: 1) The '50% off' claim needs..."
```

The tradeoff: More thinking = better answers, but slower and uses more of max_tokens.

Budget	Behavior	Use Case
0	No thinking at all	Fast tasks: routing, short copy
512	Brief reasoning	Content writing, SEO
1024	Moderate reasoning	Compliance, analytics, UI code
2048	Deep reasoning	Orchestration, backend architecture

*IMPORTANT: **max_tokens** includes BOTH thinking AND answer tokens. Formula: **max_tokens = thinking_budget + expected_output***



Agent Assignment

[Quality] Server (9B, port 8090) — 7 agents

Tasks where **quality matters more than speed**. The 9B model thinks before answering.

#	Agent	Think Budget	Output	max_tokens	Role
1	main (orchestrator)	2048	2048	4096	Routes & coordinates
4	campaign-strategist	1024	3072	4096	Strategy & positioning
7	seo-article-writer	512	6144	8192	Long-form SEO articles
10	compliance-qa	1024	2048	3072	Policy & compliance
11	analytics-optimizer	512	2048	3072	Performance analysis
12	ui_engineer	1024	8192	9216	Frontend/UI code
13	backend_architect	2048	8192	10240	Backend & API code

** Code agents get larger budgets: code generation needs more thinking to plan the approach and larger output because a single component can be 200-400 lines.*

■ Speed Server (2B, port 8091) — 7 agents

Tasks where **speed matters more than depth**. No thinking overhead.

#	Agent	Thinking	max_tokens	Role
2	trend-scout	OFF	2048	Trends & market signals
3	offer-hunter	OFF	2048	Competitive deals
5	shortscript-writer	OFF	2048	Short-form scripts
6	post-writer	OFF	2048	Social posts & copy
8	voice-agent	OFF	1024	Voice workflows
9	ugc-video-agent	OFF	2048	UGC & video concepts
14	agile_dev	OFF	4096	Bug fixes, small patches

** agile_dev stays on 2B: it handles small, scoped patches. For full features, backend_architect on 9B takes over. 2B is fast (210 t/s) keeping the dev loop tight.*

Per-Request Thinking Control

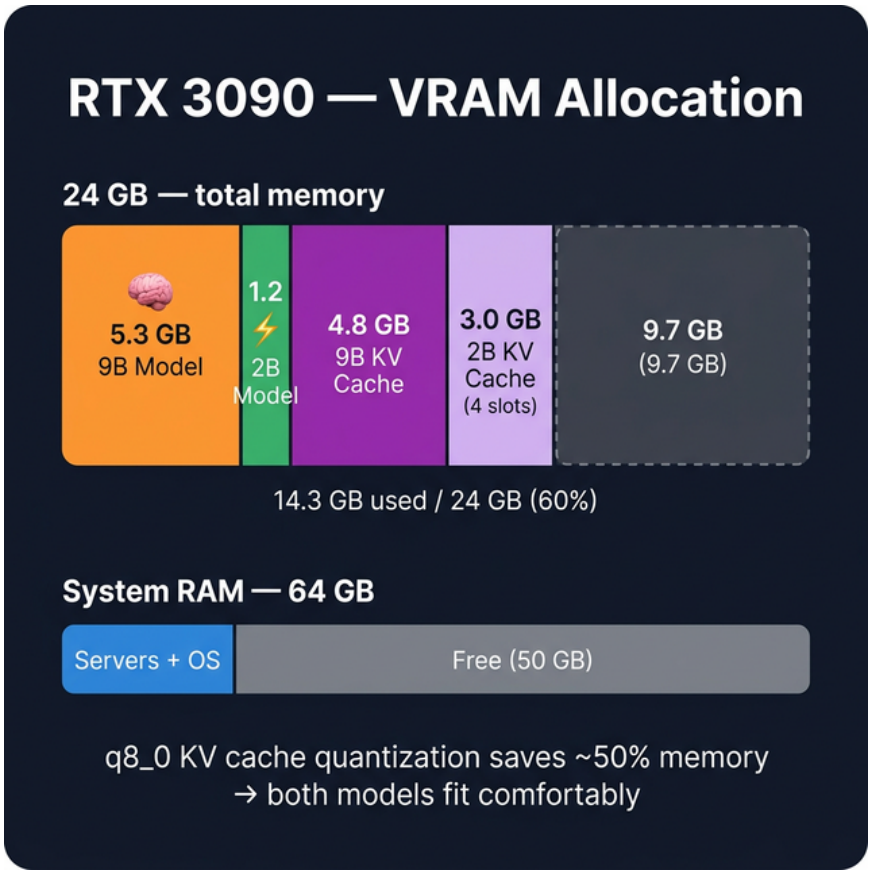
Each agent sets its own thinking budget via the API:

```
# Compliance agent (9B) — needs deep thinking POST
http://server:8090/v1/chat/completions { "reasoning_budget": 1024, ← up to 1024
thinking tokens "max_tokens": 3072 ← 1024 thinking + 2048 answer } # Post writer (2B) —
no thinking needed POST http://server:8091/v1/chat/completions { "reasoning_budget": 0,
← no thinking at all "max_tokens": 512 ← pure answer output }
```

*When the thinking budget is reached, the model is **forced to stop thinking** and immediately start producing the answer. The thinking it already did still helps — it stays in the model's context, so even partial thinking improves answer quality.*

Resource Summary

Resource	Value
GPU	RTX 3090, 24 GB VRAM
VRAM used	14.3 GB (60%)
VRAM free	10.3 GB (headroom for KV cache growth)
System RAM	64 GB (~6 GB OS + ~8 GB KV cache swap = ~14 GB used, 50 GB free)
Context window	262K tokens per server
KV cache	q8_0 (8-bit, saves ~50% VRAM vs default)
Total agents	14
Models loaded	2 (simultaneously)



VRAM breakdown: both models + KV cache fit with 40% headroom

Want the Full Implementation Guide?

This presentation showed WHAT we built. The full guide shows exactly HOW - every patch, every config flag, every optimization explained step by step.

Join the free AIgenteur community to get it:

https://www.skool.com/aigenteur-academy-9764

Inside you will find: patched llama.cpp source, custom Jinja templates, server launch scripts, budget enforcement code, and deployment configs.

AIgenteur Academy

Local AI. Real hardware. No cloud.

14 OpenClaw AI Agents Running 100% Locally on a Single RTX 3090 GPU

52ms, Not 6 Minutes

Implementation Guide

Zero cloud APIs. Zero subscriptions. Full reasoning on one GPU from 2020.

Step-by-step: from stock llama.cpp to a production multi-agent backend

6,667x faster with KV cache offloading

Andrew Darius
AIgenteur Academy
skool.com/aigenteur-academy-9764

What We Built

A dual-model LLM backend running on a single RTX 3090 (24 GB VRAM) that serves 14 AI agents simultaneously. The key innovations:

- **Two models on one GPU** - Qwen3.5-9B (quality) + Qwen3.5-2B (speed) run side-by-side
- **Per-request thinking budget** - each agent gets its own reasoning token limit
- **KV cache offloading** - system prompts cached so repeat requests skip prompt processing
- **Budget enforcement** - forced </think> injection when reasoning limit is reached
- **262K context window** - with q8_0 quantized KV cache to save ~50% VRAM

Component	What	Why
llama.cpp (patched)	Custom-built inference server	Per-request reasoning budget support
Qwen3.5-9B Q4_K_M	5.3 GB quality model	Deep reasoning for complex agents
Qwen3.5-2B Q4_K_M	1.2 GB speed model	Fast output for simple agents
Chat template override	27B Jinja template	9B GGUF template lacks enable_thinking
server-context.cpp patch	Budget tracking code	Force thinking state + close tag injection

The Thinking Mode Problem

Qwen3.5 models support a "thinking mode" where the model reasons inside <think>...</think> tags before answering. But getting this to work with per-request budgets required solving three problems:

Problem 1: Template Detection

Stock llama.cpp checks if the GGUF's embedded Jinja template supports **enable_thinking** by rendering it both ways and comparing output. The Qwen3.5-9B GGUF template is a simpler version that does NOT change output when enable_thinking is toggled - so llama.cpp concludes thinking is unsupported and disables it entirely (thinking = 0).

> *The Qwen3.5-27B GGUF has the full template with enable_thinking support. Solution: extract the 27B's template and use it for the 9B via --chat-template-file flag.*

Problem 2: Server-Level Enable

Even with the correct template, the server initialization code at **server-context.cpp line ~908** determines enable_thinking based on template detection. When reasoning_budget > 0 but the template check fails, thinking stays disabled.

> *Patch: modify the logic so that when reasoning_budget > 0, enable_thinking is forced to true regardless of template detection result. This is a one-line change in the init function.*

Problem 3: Budget Tracking

Stock llama.cpp only treats --reasoning-budget as on/off (-1 or 0). The patched version accepts positive integers as real token limits. But the budget tracker at **line ~2808** searches for <think> in generated text to know when thinking starts. Problem: the Jinja template puts <think> in the prompt, not the generated text - so the tracker never activates.

> *Patch: instead of scanning for <think> in generated text, start in REASONING_STATE_REASONING immediately on the first token when budget > 0. The template already prefixed <think>, so generated text IS thinking from token 1.*

The 5 Code Changes

All changes are in the llama.cpp source tree. Here is exactly what was modified and why. The actual code is available to AIgenteur community members.

#	File	What Changed	Lines
1	common/common.h	Added reasoning_budget, n_reasoning_tokens, reasoning state enum, forced_tokens vector	~5 new fields
2	common/arg.cpp	Added --reasoning-budget CLI flag that accepts positive integers	~15 lines
3	server-context.cpp (init)	Force enable_thinking=true when reasoning_budget > 0	1 line changed
4	server-context.cpp (generation)	Budget tracking: start in REASONING state immediately, count tokens, inject </think> when exceeded	~40 lines added
5	server-task.h/cpp	Pass reasoning_budget from API request to server slot	~10 lines

Total code changes: approximately 70 lines across 5 files. The changes are minimal and surgical - they don't alter llama.cpp's core inference path, only the server's request handling and budget enforcement.

Chat Template Override

The single most important fix was the chat template. Here's why:

	9B GGUF Template	27B GGUF Template
Size	~2 KB (simplified)	~7.7 KB (full)
enable_thinking	Not supported	Fully supported
<think> prefix	Never inserted	Inserted when thinking=true
</think> handling	No logic	Parses reasoning_content
Tool calling	Basic	Full parallel support

How to extract the template:

```
# Start the 27B model temporarily llama-server -m Qwen3.5-27B-Q4_K_M.gguf --port 8090 #
Extract template from the /props API endpoint curl -s http://127.0.0.1:8090/props | \
python3 -c "import json,sys; \ r=json.load(sys.stdin); \
open('qwen35_template.jinja','w').write(r['chat_template'])"
```

Then apply it to any Qwen3.5 model:

```
llama-server \ -m Qwen3.5-9B-Q4_K_M.gguf \ --chat-template-file qwen35_template.jinja \
--reasoning-budget 2048 \ --port 8090
```

This is the key insight: the 9B and 27B use the same architecture and tokenizer - only the GGUF-embedded template differs. Swapping it unlocks thinking for the 9B.

Production Server Configuration

9B Quality Server (port 8090)

```
llama-server \ -m Qwen3.5-9B-Q4_K_M.gguf \ --port 8090 --host 0.0.0.0 \ -ngl 99 -np 1 -c
262144 \ -ctk q8_0 -ctv q8_0 \ --flash-attn on \ --reasoning-budget 2048 \
--chat-template-file qwen35_template.jinja
```

2B Speed Server (port 8091)

```
llama-server \ -m Qwen3.5-2B-Q4_K_M.gguf \ --port 8091 --host 0.0.0.0 \ -ngl 99 -np 4 -c
262144 \ -ctk q8_0 -ctv q8_0 \ --flash-attn on \ --reasoning-budget 0 \
--chat-template-file qwen35_template.jinja
```

Flag	Purpose
-ngl 99	All layers on GPU
-np 1 / -np 4	Parallel slots (1 for 9B serial, 4 for 2B parallel)
-c 262144	262K context window
-ctk q8_0 -ctv q8_0	8-bit KV cache (saves ~50% VRAM)
--flash-attn on	Flash attention for memory efficiency
--reasoning-budget N	Default thinking budget (overridable per request)
--host 0.0.0.0	Accept connections from network

The 6,667x Speedup: KV Cache Offloading

The headline-grabbing 52ms vs 5.8 minutes comes from **KV cache caching**. Here's exactly how it works:

Without cache (cold start):

- Every request must process the FULL context: system prompt + history + new message
- A 200K token system prompt at 573 tok/s = **5.8 minutes** of prompt processing
- The GPU does 200K forward passes just to rebuild the KV cache
- Then starts generating the actual answer

With KV cache (warm):

- System prompt is processed ONCE and its KV cache is kept in memory
- Subsequent requests reuse the cached KV entries
- Only the NEW tokens (user message, ~30 tokens) need processing
- 30 tokens at 573 tok/s = **52 milliseconds**

This is automatic in llama.cpp - the server keeps a prompt cache per slot. As long as the system prompt stays the same (which it does for each agent), every request after the first one is nearly instant.

Metric	9B (no cache)	9B (cached)	2B (cached)
Prompt processing	5.8 min	52 ms	21 ms
Speedup	-	6,667x	6,667x
First token latency	~6 min	~60 ms	~25 ms
Where bottleneck is	Prompt eval	Generation speed	Generation speed

How Budget Enforcement Works Internally

When a request arrives with reasoning_budget=50:

- 1. Template prepends <think> to the assistant's response
- 2. Model starts generating thinking tokens
- 3. Budget tracker counts each generated token (n_reasoning_tokens++)
- 4. At token 50: tracker sets REASONING_STATE_FINISHED
- 5. Injects </think> as forced tokens into the generation queue
- 6. Model sees </think> and transitions to producing the answer
- 7. All 50 thinking tokens remain in KV cache - model "remembers" its reasoning

Why partial thinking helps:

The thinking tokens are part of the autoregressive context. When </think> is forced, the model generates the answer with all previous thinking tokens visible in its attention window. Even 20-50 tokens of planning significantly improves answer quality compared to zero thinking.

Per-request API:

```
POST /v1/chat/completions { "messages": [...], "reasoning_budget": 100, // override
server default "max_tokens": 512 // includes thinking + answer } Response: { "choices":
[{ "message": { "content": "The capital of France is Paris.", "reasoning_content": "1.
Identify the question..." } }] }
```

Quick Start Checklist

Prerequisites:

- NVIDIA GPU with 16+ GB VRAM (RTX 3090/4090 recommended)
- CUDA 12.x+ installed
- cmake, make, git, python3
- ~8 GB disk space for models

Steps:

- 1. Clone and patch llama.cpp (5 file changes, ~70 lines)
- 2. Build with CUDA: cmake -DGGML_CUDA=ON + make
- 3. Download Qwen3.5-9B-Q4_K_M.gguf and Qwen3.5-2B-Q4_K_M.gguf from HuggingFace (unsloth)
- 4. Extract the 27B chat template (or use the one from the community)
- 5. Launch 9B server on port 8090 with --reasoning-budget 2048
- 6. Launch 2B server on port 8091 with --reasoning-budget 0
- 7. Configure your agent framework to route to the correct port
- 8. Test: curl the /v1/chat/completions endpoint with reasoning_budget

The complete patched source code, pre-extracted chat template, launch scripts, and agent routing configuration are all available to AIgenteur community members.

Get the Complete Package

This guide explained the concepts and approach. The full package includes everything you need to replicate this setup:

- Patched llama.cpp source (all 5 files)
- Pre-extracted Qwen3.5 chat template (qwen35_template.jinja)
- Server launch scripts for dual-model setup
- Agent routing configuration for OpenClaw
- VRAM calculator spreadsheet
- Benchmark results and tuning guide

Join the free AIgenteur community:

https://www.skool.com/aigenteur-academy-9764

AIgenteur Academy

Local AI. Real hardware. No cloud.